Mechanics of Solids

Force Analysis of a Planar Truss

Aksh Solanki (24110023)
Modalavalasa Anusha (24110206)
Kavya Parmar (24110238)
Pranav Magarde (24110269)

Objective:

The goal of this project is to compute the internal forces in the members of a 2D truss under applied external loads, provided it is statically determinate. This involves determining the exact magnitude of force each member experiences while also stating whether it is in compression or tension.

A truss is a structural framework made up of slender members connected at joints. It appears in bridges, roof supports, and towers. When loads are applied, each member either pulls apart (tension) or gets squeezed together (compression). Thus knowing the value of these parameters is important for checking whether the structure is safe or not.

To do this systematically, a program was written that takes a user-defined truss geometry, support conditions, and applied loads, and then solves for all reaction forces and internal member forces using classical methods of structural analysis.

Problem Statement:

Consider a 2D truss consisting of n joints and m members. Joints are connected by 2-4 force members that can only carry axial load. External forces are applied at some specific joints, and the truss is supported at two points as decided by the problem. One of the supports is pin support and one is roller support, providing the three reaction components needed for static determinacy.

For a statically determinate truss, the following condition shall hold:

$M + r = 2j$

where m is the number of members, r is the number of reaction components (3 for a pin-roller configuration), and j is the number of joints.

The objective is to find:

1. The reaction forces at the supports (Rx, Ry at the pin; R normal to the roller surface).
2. The internal axial force in each member, along with its nature (tension or compression).

All joints are treated as frictionless pins, meaning members carry no bending moments. External loads are specified by magnitude and angle, which are then decomposed into x and y components for analysis.

Method:

The method follows the fundamental approach of solid mechanics, breaking it down into two stages:

External Equilibrium

The entire truss is treated as one rigid body. This is done so to analyse the global reaction forces acting on supports from the external environment. The three equilibrium equations for a 2D structure are:

$\sum Fx=0$, $\sum Fy=0$, $\sum Mo=0$

A pivot must be selected to calculate the moment. The program is meant to select the pin support (o) as the pivot, this eliminates its own unknown reactions from the moment equation, reducing the system to a 3×3 linear system. This is solved to get the reaction forces at both supports.

Internal Equilibrium (Method of Joints)

Once reactions are known, the program moves joint by joint. At each joint, two equilibrium equations are available ($\sum Fx=0$ and $\sum Fy=0$). The method proceeds by always selecting a joint with at most two unknown member forces. This joint is treated as a temporary origin and all known forces are equated to unknown. This gives a 2×2 linear system at each step.

Sign Convention for Tension and Compression

During the analysis, each unknown member force is initially assumed to act away from the joint (In the direction of the member, thus tension assumption). After solving, a positive result confirms tension; a negative result means the force actually acts toward the joint, so the member is in compression.

Numerical Implementation:

The program is written in Python and structured around four input phases followed by a solve phase.

Phase 1: Defining the Geometry (Joints)

The first step is to establish the physical shape and dimensions of the truss.

1. **Setting the Grid:** The program operates on a virtual 2D canvas defined by an X and Y axis. Users are instructed to place the leftmost point of their truss at the origin (0, 0).
2. **Inputting Coordinates:** The user inputs the physical location of every joint in meters (e.g., X=3, Y=4). The user can also assign each joint a label (e.g., "A", "B", "C").
3. **Storing the Data (Hash Maps):** The program stores these coordinates in a **Dictionary** (Hash Map). It creates two distinct dictionaries:
    - One linking a unique ID number to the (X, Y) coordinates.
    - One linking the ID number to the alphabetical label.

There is also an auto-scaling feature wherein if any joint exceeds 10m limit, the coordinates are scaled down by a factor of 10 till the truss fits perfectly within the 20x20 display grid.

Phase 2: Building the Connections (Force Members)

Once the joints exist in space, the user defines the beams connecting them.

1. **Defining Connections:** The user types the two labels of the joints they wish to connect (e.g., "A B").
2. **Dictionary Lookup:** The program utilizes a reversed dictionary (mapping letters back to ID numbers). This allows it to instantly verify if the user's input matches existing joints without having to scan through an array.

Validation: The program checks for errors. It ensures that the joint labels actually exist, prevents a user from connecting a joint to itself (e.g., "A A"), and blocks the creation of duplicate members.

Phase 3: Grounding the Truss (The Supports)

The truss has to be anchored to the surroundings using supports.

1. **Selecting Support Types:** The user can assign following supports to specific joints.
    - **Pin Support:** Fixes the joint in both the X and Y directions.
    - **Roller Support:** Allows sliding along a surface but prevents movement perpendicular to it. Users can also define the angle roller has been kept.

Static Determinate Analysis: The system monitors the total number of reaction unknowns. For a statically determinate 2D truss, three independent reaction components are required. The program notifies the user if the support configuration fails to meet this equilibrium criteria.

Phase 4: Applying the Forces (The Loads)

Finally, the external weights or forces are applied to the structure.

1. **Polar Input:** The user inputs loads using polar coordinates: a Magnitude (in kilonewtons, kN) and an Angle (in degrees).
2. **Vector Decomposition:** The software decomposes polar force inputs into orthogonal X and Y components using trigonometric principles. These resultant force vectors are then applied to the designated nodes within the mathematical model.

Once these four phases are complete, the program has all the mathematical data it needs to begin solving for the reaction forces and internal member forces.

Phase 5: Solving the Truss

Computing the Global Reactions: The truss is first treated as a one single rigid body. Three equilibrium equations $\sum Fx=0$, $\sum Fy=0$, and $\sum M=0$ for it are assembled into a 3×3 coefficient matrix. The moment equation is written about the pin support, thus its unknown forces are removed from the calculations. The right-hand side vector holds the negated applied force components. numpy.linalg.solve returns the three reaction unknowns in one call.

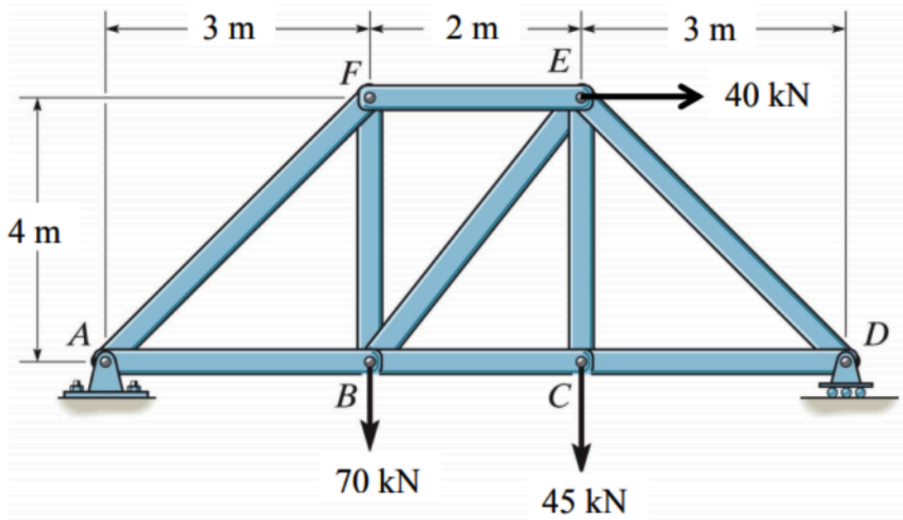
Method of Joints Loop: With reactions known, the program enters a while loop that runs until all members are solved. Each iteration scans the joint list for any joint where the count of unsolved connected members is ≤ 2. When one is found, the program computes a unit vector for each unknown member by subtracting the current joint's coordinates from the far-end joint's coordinates and normalizing. This directly gives the x and y direction cosines without ever computing an explicit angle. Known forces at joints like reactions, applied loads, or previously solved member forces are summed and moved to the right-hand side. Overall, the two equilibrium equations form a 2×2 system, solved again with numpy.linalg.solve.

Tension/Compression Classification: The solver assumes all unknown member forces act outward (tension). A positive result confirms this; a negative result means the force acts inward, so the member is in compression. After each joint is solved, its members are flagged as known and their forces are stored in a dictionary for reuse at adjacent joints. The loop continues until the dictionary is fully filled.

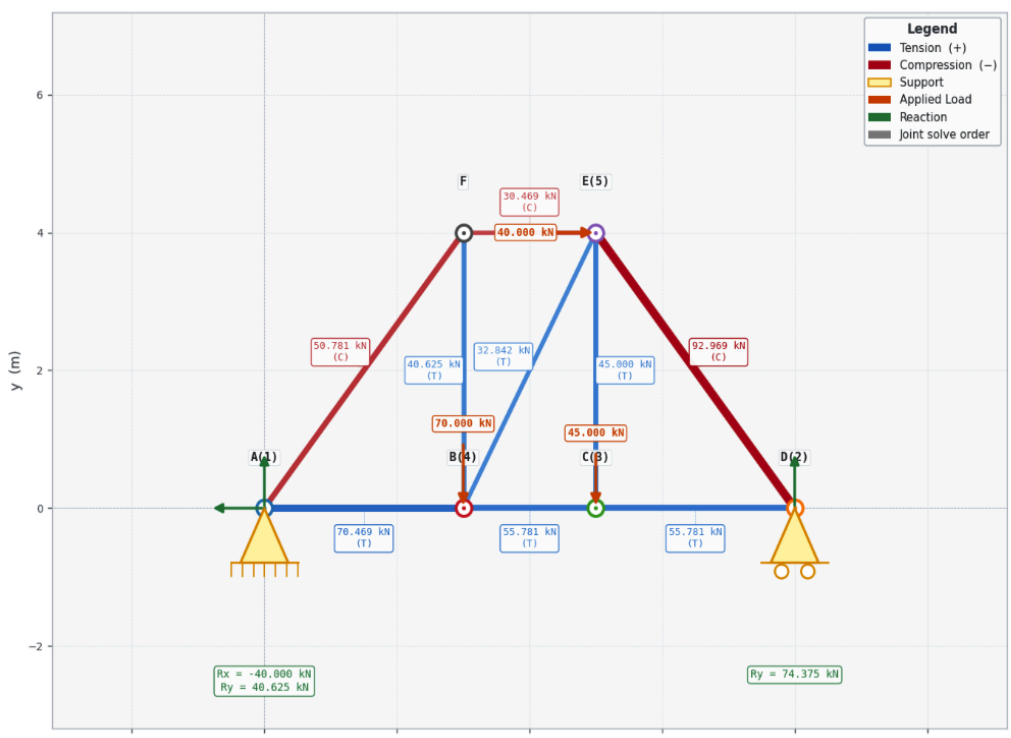
The loop repeats marking solved members as known, using their forces at adjacent joints, until every member is resolved.

Results and Discussion:

The program was tested on the given problem:



There is a pinned and roller support to support the whole system of truss. The axial forces applied are: In the horizontal axis 40 kN, and in the vertical axis, 70 kN and 45 kN acting downwards as shown in the figure. The whole 2D truss is supported by 6 joints and 9 force members.



The system is color-coded to distinguish forces: blue members indicate tension, red members indicate compression, orange arrows show external loads, and green arrows represent reaction forces. Joints are labeled (A, B, C, etc.) with numbers in brackets indicating the order in which they were solved.

```
=====================================================================
                    2D TRUSS  -  METHOD OF JOINTS
=====================================================================

  REACTIONS
  ------------------------------------------------
    Joint A:  Rx =   -40.000 kN,  Ry =    40.625 kN
    Joint D:  Rx =     0.000 kN,  Ry =    74.375 kN

=====================================================================
  MEMBER FORCES
---------------------------------------------------------------------
    #    Member       L (m)      Force (kN)     Nature
---------------------------------------------------------------------
    0     A-B         3.000       70.469 kN     TENSION
    1     B-C         2.000       55.781 kN     TENSION
    2     C-D         3.000       55.781 kN     TENSION
    3     D-E         5.000      -92.969 kN   COMPRESSION
    4     F-E         2.000      -30.469 kN   COMPRESSION
    5     A-F         5.000      -50.781 kN   COMPRESSION
    6     F-B         4.000       40.625 kN     TENSION
    7     E-C         4.000       45.000 kN     TENSION
    8     B-E         4.472       32.842 kN     TENSION
=====================================================================
```

These are the forces along with their respective natures for each member in the truss.

Learning Outcomes:

This project helped us connect and implement the textbook knowledge of solid mechanics to computations. Writing equilibrium equations by hand for a few joints is one thing, getting a program to do it for any arbitrary truss required critical thinking. We had to use the available information to find the unknown and perform the necessary operations in an ordered way.

The method of joints, in particular, taught me that the sequence of solving matters, you cannot jump to a joint with three unknowns just because you want to. That constraint forced a structured, iterative approach that mirrors how one would solve it by hand on paper.

On the computational side, working with numpy.linalg.solve gave practical exposure to how linear systems are solved in code, and why approaching these systems with matrix methods are so much more scalable and solvable than directly trying to manipulate the equations symbolically.

A solver that crashes on bad input is not useful. Building preliminary checks like catching duplicate members, checking static determinacy made the program more robust. This made us write a code that fails gracefully rather than just a code that works on clean data.